POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

SYLABUS PRZEDMIOTU

Nazwa przedmiotu:	**Warsztat Programisty**
Kod przedmiotu:	WPR
Kierunek / Profil:	Informatyka / praktyczny
Tryb studiów:	niestacjonarny
Rok / Semestr:	1 / 1
Charakter:	obowiązkowy
Odpowiedzialny:	mgr inż. Adam Urbanowicz
Wersja z dnia:	16.03.2026

1. Godziny zajęć i punkty ECTS

Wykłady	Ćwiczenia	Laboratori	Z prowadzącym	Praca własna	Łącznie	ECTS
—	—	24 h	24 h	51 h	75 h	**3**

2. Forma zajęć

Forma zajęć	Sposób zaliczenia
Laboratorium	Zaliczenie z oceną

3. Cel dydaktyczny

Celem przedmiotu jest wykształcenie u studentów praktycznych umiejętności pracy warsztatowej programisty, w szczególności w zakresie korzystania z narzędzi programistycznych, debugowania kodu, pracy zespołowej z wykorzystaniem systemu kontroli wersji Git oraz platformy GitHub. Zajęcia mają na celu zapoznanie studentów z podstawami automatyzacji procesów (CI/CD), debugowania aplikacji internetowych z użyciem narzędzi deweloperskich przeglądarek, podstawami technologii webowych (HTML, CSS, JavaScript), a także wprowadzenie do świadomego i odpowiedzialnego wykorzystania narzędzi opartych na sztucznej

inteligencji w pracy programisty. Dodatkowym celem jest przygotowanie studentów do budowania własnego portfolio programistycznego.

4. Treści programowe

1. 'nr_zajec': 1, 'wyklad': 'Organizacja pracy programisty: repozytorium kursowe, wymagane konta (GitHub), struktura materiałów, zasady oddawania prac i checklisty jakości.', 'cwiczenia': ''
2. 'nr_zajec': 2, 'wyklad': 'Przegląd popularnych IDE i narzędzi: Visual Studio / VS Code / IntelliJ / Rider (porównanie), konfiguracja środowiska, rozszerzenia, formatowanie i linting.', 'cwiczenia': ''
3. 'nr_zajec': 3, 'wyklad': 'Debugowanie kodu: podstawy debuggera (breakpointy, step in/out/over, watch, call stack), praca na przykładach w aplikacji konsolowej.', 'cwiczenia': ''
4. 'nr_zajec': 4, 'wyklad': 'Debugowanie błędów wykonania: wyjątki, logowanie, asercje, minimalny repro-case, czytanie stack trace, kultura zgłaszania błędów (issue).', 'cwiczenia': ''
5. 'nr_zajec': 5, 'wyklad': 'Git: model pracy i podstawowe komendy (init/clone/status/add/commit/log ignorowanie plików (.gitignore), dobre komunikaty commitów.', 'cwiczenia': ''
6. 'nr_zajec': 6, 'wyklad': 'Git: praca z repozytorium zdalnym (remote, fetch, pull, push), rozwiązywanie typowych problemów, praca w parach i code review (wstęp).', 'cwiczenia': ''
7. 'nr_zajec': 7, 'wyklad': 'Branche w Git: tworzenie i przełączanie gałęzi, merge vs rebase (intuicja), konflikty i ich rozwiązywanie na prostych przykładach.', 'cwiczenia': ''
8. 'nr_zajec': 8, 'wyklad': 'Pull Requesty na GitHub: workflow PR, szablon opisu, review, komentarze, poprawki, podstawy pracy z Issues i Projects (tablica zadań).', 'cwiczenia': ''
9. 'nr_zajec': 9, 'wyklad': 'Wprowadzenie do prostych pipeline'ów: czym jest CI/CD, podstawowy workflow (np. build/test/lint), plik konfiguracyjny, uruchomienia automatyczne po push/PR.', 'cwiczenia': ''
10. 'nr_zajec': 10, 'wyklad': ' Publikacja statycznej strony na GitHub Pages: struktura repo, branch/folder publikacji, podstawy konfiguracji, dodanie prostej strony kursowej.', 'cwiczenia': ''
11. 'nr_zajec': 11, 'wyklad': ' DevTools w przeglądarce: inspekcja DOM/CSS, podgląd i modyfikacja stylów na żywo, box model, debugowanie responsywności.', 'cwiczenia': ''
12. 'nr_zajec': 12, 'wyklad': ' DevTools zaawansowane: zakładka Network (żądania/odpowiedzi), Console (logowanie, błędy), Sources (breakpointy w JS), podstawy wydajności (Performance/Lighthouse).', 'cwiczenia': ''
13. 'nr_zajec': 13, 'wyklad': ' Wstęp do HTML: struktura dokumentu, semantyka, formularze (podstawy), dostępność (a11y) i dobre praktyki.', 'cwiczenia': ''
14. 'nr_zajec': 14, 'wyklad': ' Wstęp do CSS: selektory, kaskada i specyficzność, flexbox/grid (wprowadzenie), responsywność, organizacja stylów w projekcie.', 'cwiczenia': ''

15. 'nr_zajec': 15, 'wyklad': ' Wstęp do JavaScript: typy i zmienne, funkcje, zdarzenia w DOM, proste operacje na elementach strony; wstęp do pracy z agentem AI w roli asystenta programisty (promptowanie, weryfikacja odpowiedzi, zasady bezpieczeństwa i etyki).', 'cwiczenia': ''

5. Efekty kształcenia

Wiedza

- 'keu': 'K_W10', 'peu': 'Student potrafi czytać ze zrozumieniem kod języków skryptowych, tworzyć skrypty oraz zweryfikować poprawność utworzonego kodu.', 'metoda_weryfikacji': ''
- 'keu': 'K_W21', 'peu': 'Student potrafi projektować aplikacje zgodnie z paradygmatem strukturalnym i obiektowym. Student posiada podstawową wiedzę na temat protokołu HTTP oraz tworzenia bezpiecznych aplikacji internetowych i bazodanowych.', 'metoda_weryfikacji': ''
- 'keu': 'K_W10', 'peu': 'Student zna odpowiednie narzędzia wspomagające tworzenie skryptów oraz wspomagające projektowanie aplikacji internetowych, bazodanowych', 'metoda_weryfikacji': ''

Umiejętności

- 'keu': 'K_U19', 'peu': 'Student potrafi wybrać odpowiednie środowisko programistyczne i inne narzędzia, które wspomagają w projektowaniu aplikacji oraz potrafi dobrać model procesu wytwarzania aplikacji do specyfiki przedsięwzięcia. Student potrafi posługiwać się wybranym środowiskiem oraz analizować poprawność działania stworzonej aplikacji.', 'metoda_weryfikacji': 'kolokwium, ocena sporządzonego oprogramowania, prezentacja projektu i dokumentacji'
- 'keu': 'K_U16', 'peu': 'Student potrafi wykorzystać odpowiednie biblioteki do tworzenia aplikacji oraz wykorzystać narzędzia, które umożliwiają tworzenie, debbugowanie i uruchomienie projektów programistycznych. Student potrafi przedstawić stworzoną aplikację oraz omówić jej sposób działania.', 'metoda_weryfikacji': 'kolokwium, ocena sporządzonego oprogramowania, prezentacja projektu i dokumentacji'
- 'keu': 'K_U27', 'peu': 'Student potrafi skonfigurować środowisko potrzebne do stworzenia skryptów i aplikacji internetowej i bazodanowej, tzn. posiada wiedzę na temat konfiguracji bazy danych oraz oprogramowania, które jest potrzebne do tworzenie projektu programistycznego.', 'metoda_weryfikacji': 'kolokwium, ocena sporządzonego oprogramowania, prezentacja projektu i dokumentacji'

6. Kryteria oceny

- wyklad
- cwiczenia_laboratorium

7. Metody dydaktyczne

cwiczenia_laboratorium:

- rozwiązywanie zadań
- metoda projektów (projekt praktyczny)
- warsztaty

8. Literatura

Podstawowa:

- Dokumentacja języka: www.php.net
- w3schools.com
- Larry Ullman ,PHP i MySQL. Dynamiczne strony WWW. Szybki start, Helion 2018

Uzupełniająca:

- Brak danych.